

# Why MLOps? (The “why now” story)

- **Problem in the real world:** Lots of models die in notebooks and never reach users. MLOps emerged to fix that delivery gap by **automating** the path from data → model → production, borrowing practices from DevOps. If it isn't automated, it's fragile.
  - **Core idea:** MLOps = **Data Science + DevOps** (experiments + reliable software delivery).
  - **What “Ops” adds to ML:**
    - **Continuous Integration (CI):** run automated tests on code and pipelines whenever you push, using tools like GitHub Actions/Jenkins/etc.
    - **Continuous Delivery (CD):** automatically ship to environments using **Infrastructure as Code (IaC)** so infra changes are versioned and reproducible.
    - **Microservices & Serverless:** small, deployable prediction services (e.g., Flask API on Cloud Run/Fargate/Azure App Services) or Lambda functions.
    - **IaC:** declare cloud resources in code (Terraform/CloudFormation) → consistent, repeatable environments.
    - **Monitoring & instrumentation:** collect telemetry (latency, errors, drift signals) and improve continuously (Kaizen mindset).
- 

## The ML lifecycle: data → training → tracking → registry → deployment → monitoring

Below is a clean, exam-ready lifecycle with the **key artifacts, tools, and risks** at each step.

### 1. Data & Features

- Build and reuse standardized features in a **Feature Store** to avoid “Team A computed CLV one way, Team B another way.” This prevents **train-serve skew** (mismatch between training features and live features). Feature stores also surface features for discovery and support drift monitoring.
- Cloud offerings: **Vertex AI Feature Store, SageMaker Feature Store, Azure ML Feature Store (preview)**.

## 2. Experimentation & Training

- Track code, data versions, parameters, metrics, and artifacts so results are **reproducible**.
- Use **CI** to test data/feature code and training code continuously.
- Cloud training options include managed GPU/TPU services (e.g., **Vertex AI Training / TPUs, Azure ML Compute, AWS EC2/EKS/ECS**).

## 3. Experiment Tracking (e.g., MLflow Experiments)

- Every run logs **params, metrics, tags, artifacts** to a tracking server/URI; this is the foundation for selecting a champion model later. (We’ll detail MLflow in its own topic; this step connects the lifecycle.)

## 4. Model Registry

- A central place to **register versions** of models (e.g., “v1 Staging → v1 Production”), add lineage, approvals, and rollbacks.
- The cloud stacks pair registry with endpoints and CI/CD (e.g., **Vertex AI Experiments/Models + Endpoints, SageMaker Experiments + Endpoints, Azure ML Experiments + Endpoints**).

## 5. Deployment (real-time / batch / streaming)

- Package inference code as a **microservice** (container or serverless). Patterns like **blue/green, canary, shadow** are used to reduce risk when promoting new models.
- Managed options: **Vertex AI Endpoints, Cloud Run, GKE Autopilot, SageMaker Real-time/Async, Azure Managed Online Endpoints / AKS**.

## 6. Production Monitoring (quality + ops)

- **App health:** latency, throughput, error rates.
- **Model health:** drift in input data distribution or target concept; keep a feedback loop. Tools include **Vertex Model Monitoring**, **SageMaker Model Monitor**, **Azure Data Drift monitor**, plus cloud logging/metrics.
- **Continuous improvement:** telemetry → tickets/experiments → retrain (Kaizen).

**Why this lifecycle is necessary:** Without common features, tracking, and a registry, you can't reproduce or safely promote models. Without CI/CD/IaC, infra drifts. Without monitoring, performance silently decays.

---

## Where containers/Kubernetes fit (quick link to later topics)

- **Containers** package code + deps → fast startup, portable. **Kubernetes** orchestrates and scales horizontally to meet variable load; vertical scaling means giving one pod more CPU/RAM.
  - You'll use them when turning a trained model into a **production microservice** behind an API. (We'll go deep when we hit the Docker/K8s topics.)
- 

## How clouds map to the lifecycle (you'll likely get comparison Qs)

- **Compute/Train/Serve:** EC2/EKS/ECS/Fargate vs AKS/Container Apps vs GKE/Cloud Run.
- **Feature stores:** SageMaker FS vs Azure ML FS (preview) vs Vertex AI FS.
- **Experiments/Deployment/Monitoring:** SageMaker Experiments/Endpoints/Model Monitor, Azure ML Experiments/Online Endpoints/Data Drift Monitor, Vertex AI

Experiments/Endpoints/Model Monitoring.

---

## Fast recap (2–3 lines)

MLOps = **automated, reproducible ML** using DevOps ideas (CI/CD, IaC, microservices, monitoring).

The lifecycle runs **Data/Features** → **Training/Tracking** → **Registry** → **Safe Deployment** → **Monitoring** → **Continuous improvement**.

Clouds provide end-to-end primitives (feature stores, experiments, endpoints, monitors) to implement this reliably.

---

## MCQs (with answers + explanations)

1. **Which best captures the goal of MLOps?**

- A. Train bigger models only
- B. Automate and operationalize the ML lifecycle end-to-end
- C. Replace data scientists with AutoML
- D. Avoid version control to move faster

**Answer: B.** MLOps emphasizes automation/DevOps practices across the ML lifecycle to reliably ship models.

2. **Which practice ensures infra is reproducible across environments?**

- A. Jupyter widgets
- B. Infrastructure as Code
- C. Manual runbooks
- D. Model registry

**Answer: B.** IaC declares infra in code for repeatable, idempotent deploys.

3. **What is the primary purpose of a Feature Store?**

- A. Storing BI dashboards
- B. Serving static web assets
- C. Standardizing features for training/serving to avoid train-serve skew
- D. Caching compiled binaries

**Answer: C.** Feature Stores prevent inconsistent feature definitions and train-serve skew.

4. **Which pairing is MOST aligned with CI?**

- A. Canary deploys

- B. Automated unit/integration tests on push
- C. Blue/green
- D. Manual QA

**Answer: B.** CI = continuous, automated testing of code/pipelines.

5. **A team promotes a model to production with approval gates, version tags, and rollback. Which component is central?**

- A. Data Lake
- B. Model Registry
- C. Notebook Server
- D. Scheduler

**Answer: B.** Registries manage versions/stages and enable promotion/rollback.

6. **Which cloud grouping is accurate for serving managed real-time endpoints?**

- A. S3 / BigQuery / Synapse
- B. Vertex AI Endpoints / SageMaker Real-Time / Azure Managed Online Endpoints
- C. Glue / Dataflow / Data Factory
- D. Lambda / Functions / Cloud Functions only

**Answer: B.** These are the managed model-serving endpoints in GCP/AWS/Azure.

7. **A company ships a new model to 10% of traffic to test safety before full rollout. Which pattern is this?**

- A. Blue/green
- B. Shadow
- C. Canary
- D. Rolling reboot

**Answer: C.** Canary = limited, incremental exposure. (Patterns shown in the sample architecture.)

8. **Which statement about monitoring is most correct?**

- A. Only infra metrics matter
- B. Monitoring collects app and model behavior to drive continuous improvement
- C. If CI/CD is set up, monitoring is unnecessary
- D. Monitoring is manual log reading

**Answer: B.** Monitoring (latency/errors/drift) + Kaizen → iterative improvements.

9. **Train-serve skew occurs when...**

- A. The model overfits training data
- B. Live features differ from training features
- C. Data labeling is noisy
- D. Hyperparameters are tuned poorly

**Answer: B.** Feature Store + consistent transforms mitigate skew.

10. Which is the correct map for “DevOps concept → MLOps application”?

- A. IaC → Feature engineering
- B. CI → Continuous testing of code/data pipelines
- C. CD → Manual promotion
- D. Monitoring → Occasional ad-hoc checks

**Answer: B.** CI = automated, continuous tests; CD = automated deployments; monitoring is systematic.

11. Which cloud tool is a Feature Store?

- A. Cloud Run
- B. Vertex AI Feature Store
- C. Cloud Functions
- D. Cloud Build

**Answer: B.** It's the GCP feature store option. (Alternatives: SageMaker FS, Azure ML FS.)

12. Which choice best describes a microservice in MLOps?

- A. A monolithic app with internal endpoints
- B. A small, independently deployable prediction API (often containerized)
- C. A training notebook
- D. A batch ETL job

**Answer: B.** Microservices are small, independent services; perfect for prediction endpoints.

13. Which pairing is correct for cloud monitoring of models?

- A. CloudWatch / Vertex Model Monitoring / Azure Data Drift Monitor
- B. S3 Events / BigLake / Azure Files
- C. KMS / Key Vault / Cloud KMS
- D. IAM / RBAC / IAM

**Answer: A.** These are model/inference monitoring tools in the cloud stacks.

14. Which step turns experiments into governed, versioned assets ready for promotion?

- A. Feature Store
- B. Registry
- C. Dockerfile
- D. Notebook

**Answer: B.** The registry manages model versions/stages.

15. Which concept most directly addresses environment drift across dev/stage/prod?

- A. Git LFS
- B. IaC + CI/CD
- C. Data Visualization
- D. On-prem GPUs

**Answer: B.** Version infra + automated pipelines = consistent environments.

---

## Short answers (2–3 sentences each, with key points bolded)

1. **Why do organizations adopt MLOps instead of ad-hoc ML?**  
Because manual, one-off steps don't scale or reproduce. MLOps adds automation (CI/CD), IaC, registries, and monitoring so models can be shipped safely and iterated quickly.
2. **What is a Feature Store and why is it crucial?**  
It's a **central registry of features** used in both training and serving. It prevents **train-serve skew** and accelerates reuse/discovery of features.
3. **Differentiate CI vs CD in MLOps.**  
**CI** = automated tests on commits to keep code/data pipelines healthy. **CD** = automated promotion/deployment to environments using **IaC**.
4. **Where does the Model Registry sit in the lifecycle?**  
After training/tracking, models are **registered** with versions and stages (Staging/Production). It enables **approvals, rollbacks, and lineage** in deployment.
5. **Why is monitoring not optional?**  
Production changes (data, users) cause **drift** and reliability issues. Monitoring app + model signals enables **Kaizen-style continuous improvement**.
6. **Give one cloud mapping of the same step.**  
For serving: **Vertex AI Endpoints / SageMaker Real-Time / Azure Managed Online Endpoints** are managed hyperscaler options.
7. **What deployment pattern reduces risk when rolling out a new model?**  
**Canary** or **blue/green**. They limit blast radius and allow fast rollback.
8. **How does IaC help an ML team?**  
Infra is **declared in code**, versioned, and **idempotent** across environments—no snowflake servers.
9. **What's the link between microservices and containers?**  
A model **microservice** is commonly packaged as a **container** for portability and fast

startup, and orchestrated by **Kubernetes** for scale.

10. **Why is the Feature Store specifically called out in cloud comparison tables?**

Because it's a **first-class managed component** (SageMaker/Vertex/Azure) that underpins consistent features across train/serve and accelerates productionization.